FedShieldV2

Problem Statement & Project Scoping Report

Autonomous Cross-Branch Money-Laundering Detection using Agentic AI, Graph Tracing & Federated Learning

Karthik Pragada

Executive Summary

FedShieldV2 is an autonomous, privacy-preserving anti-money-laundering (AML) system built for a single US-based bank with 3 branches. Its core innovation is a **closed agentic loop**: AI agents don't just score transactions — they investigate them, build an evidence-backed case by tracing money across accounts and branches, adversarially cross-examine their own conclusions before trusting them, and feed confirmed verdicts back into the machine learning model as fresh training labels — all without any branch ever sharing raw customer data with another.

The system targets **structuring and layering** — the classic money-laundering pattern where a criminal splits a large sum into deposits just under the $10,000 mandatory cash-reporting threshold, then moves it through a chain of accounts specifically designed to look like ordinary banking activity before pulling it back out as clean cash.

Key claim: three branches, each seeing only its own slice of a coordinated laundering scheme, independently flag suspicious activity that alone looks borderline — a shared, privacy-preserving graph then traces the money trail across all three, an AI agent pipeline builds and cross-examines the case, and a confirmed verdict automatically becomes a new training example that makes every branch's model smarter for next time.

1. Problem Statement

Multi-branch banks cannot detect coordinated cross-branch money laundering in real time, because transaction data is siloed within each branch's own systems, and regulatory and architectural constraints prevent freely pooling raw customer data across branches to look for patterns that only become visible when you connect the dots.

This structural gap is exactly what a specific laundering technique — **structuring/layering** — is designed to exploit. Each individual transaction, viewed alone, looks unremarkable. The transaction *pattern*, viewed across branches and across time, is not. No single branch has enough information to raise an alarm. The scheme succeeds by design, not by sophistication.

2. Business Problem

2.1 What Is Structuring / Layering?

Structuring (also called "smurfing") is the practice of splitting a large sum of money into multiple smaller transactions, each deliberately kept under a regulatory reporting threshold, to avoid detection. Layering is the follow-on step: moving that money through a chain of intermediate accounts to obscure its origin before it's pulled back out ("integration").

Concrete example (FedShieldV2's modeled scenario):

- Three unrelated people, each with a clean account history, each deposit cash **just under $10,000** — the real US CTR (Currency Transaction Report) threshold — at three *different* branches, within a short window of each other.
- Each deposit then moves through a chain of several intermediate accounts (the chain length is configurable — 2, 4, 6+ hops — to test whether detection holds up as the trail gets longer).

- All three chains converge on **one shared consolidation account**, which then withdraws the combined total in cash.

Why it works: each branch's local fraud model sees one large-but-not-extreme cash deposit and scores it moderately — not alarming enough alone to escalate. No branch can see that its deposit is one of three identical deposits happening at the same time elsewhere, or that the money funneling out of a dozen unrelated-looking accounts all lands in the same place.

2.2 Why Existing Systems Fail

- **Branch data silos.** Transaction history and fraud scores live locally per branch, with no real-time cross-branch feed.
- **Single-transaction thresholds.** Rules fire on individual amounts; a transfer well under a reporting threshold looks routine in isolation, regardless of how many similar transfers are happening elsewhere at the same moment.
- **No relational reasoning.** Existing systems generally don't ask "does this transaction connect to any other flagged transaction, several hops and several branches away?" — that requires a graph, not a rule engine.

2.3 Why This Can't Just Be Solved By Sharing All the Data

Regulatory and architectural reality prevents pooling raw transaction data across branches freely — whether due to formal cross-border/regulatory data-sharing constraints, or (just as commonly, even within a single institution) legacy IT fragmentation between branch systems that were never built to talk to each other in real time. FedShieldV2's design takes this as a hard constraint, not a simplification: **no branch's raw transaction data, customer name, or account number is ever visible to another branch, to the shared graph, or to any AI agent.** Only irreversibly anonymized tokens and non-identifying transaction metadata (amount, time, channel) ever leave a branch's local boundary.

2.4 Who Is Affected

- **Fraud/AML investigation teams** — currently required to manually trace money across systems, a slow, effort-intensive process. FedShieldV2 automates evidence-gathering and produces a drafted investigation report.
- **Compliance teams** — responsible for filing timely Suspicious Activity Reports; faster, automated case-building reduces regulatory risk from delayed detection.
- **The bank and its customers** — faster detection limits financial exposure and reputational risk from enabling laundering, even unknowingly.

2.5 Current State vs. Desired State

Dimension	Current State (Typical)	Desired State (FedShieldV2)
Cross-branch visibility	None — siloed per branch	Real-time shared graph tracing anonymized money flow
Detection unit	Single transaction, single branch	Multi-hop, multi-branch evidence chain
Investigation effort	Manual link analysis	Autonomous agent pipeline builds the case
Verdict quality control	Single analyst judgment call	Adversarial multi-agent cross-examination before a verdict is trusted
Model improvement	Periodic manual retraining on stale/delayed labels	Confirmed verdicts become fresh training labels automatically, continuously
Data sharing	Raw data sharing infeasible/prohibited	Only anonymized tokens + model weights ever leave a branch — never raw data

3. What Makes FedShieldV2 Novel (Research Contributions)

This is not "several AI/ML tools used together." The specific, defensible claim is a **closed loop** that doesn't exist in typical fraud-detection literature or tooling:

1. **Agent-verified active learning for federated AML.** Real AML labels are scarce and take months to arrive in the real world (formal case resolution, SAR filings). Here, an agent-driven investigation's live, evidence-backed verdict becomes a fresh training label — fed back into the model immediately, without waiting for external label sources — closing the label-scarcity gap while never requiring any branch to share raw transaction data to do it. 2. **Adversarial multi-agent verification before trusting a label.** Rather than one model or one agent unilaterally deciding "this is fraud," a Prosecutor/Defense/Judge structure argues both directions of the case before a verdict is trusted enough to feed back into training — reducing the risk of confidently-wrong labels poisoning the model over time.

4. Data Architecture

4.1 Why Synthetic Data

Real customer transaction data is not usable for this kind of build — it's PII-protected, requires governance approvals far outside project scope, and creates a compliance liability the moment it touches a development environment. The alternative used here: **statistically realistic, schema-correct synthetic data with zero real personal information**, generated with Faker (US locale).

Sufficiency argument: the model's job is to learn the *shape* of structuring (amount near a threshold, cash, timing, direction) — that shape doesn't require real names to be learned or tested meaningfully.

4.2 The Two-Sided Schema (PII Side vs. Model-Facing Side)

Every transaction is generated with a full realistic PII-bearing shape (customer identifiers, account numbers, telemetry) and immediately passed through a masking boundary before anything else in the system ever sees it:

Raw side (never leaves a branch):

- `customer_id` (synthetic, e.g. `CUST-90000001`), `ssn_last4` (fake, last-4-only)
- `account_number` (10-digit), `routing_number` (9-digit ABA, one per branch)
- Full nested transaction event: amount, type (`CASH_DEPOSIT`/`WIRE`/`ACH`/`ZELLE`/`CHECK`/`DEBIT_CARD`/`CASH_WITHDRAWAL`), channel, timestamp, originator/beneficiary blocks, telemetry (IP/device/location — generated for realism, reserved for future work, not consumed by the current detection pipeline)

Masked side (the only thing any model, graph, or agent ever sees):

- `token_id = SHA256(account_number + GLOBAL_SALT)[:16]` — one global salt across the whole system (a deliberate choice: this guarantees the same real account always maps to the same token no matter which branch processes it, which is what makes cross-branch graph tracing possible at all)
- 5 engineered features per transaction (see §4.4)
- Amount, transaction type, channel, hour, day — retained as non-identifying metadata

4.3 Fraud Injection Parameters

Parameter	Normal Range	Fraud (Structuring/Layering) Value	Why
Placement amount	Realistic transaction-size distribution	$8,000 – $9,800	Just under the $10,000 CTR threshold
Transaction type	ACH / WIRE / ZELLE / CHECK / DEBIT_CARD	CASH_DEPOSIT (placement), then WIRE/ACH (layering)	Cash is the anonymous entry point; transfers move it without re-triggering cash reporting
Channel	ONLINE / MOBILE / ATM	BRANCH (placement/exit)	Physical cash requires a branch visit
Hop count	n/a	Configurable — 2, 4, 6, or more	Tests whether detection degrades as the trail gets longer
Timing	Spread across a multi-day window	All 3 placements within ~40 minutes of each other	The coordinated signature that makes this "one scheme," not 3 coincidences
Ground truth	n/a	Logged separately, never exposed to any model/agent	Used only for offline evaluation, never for detection itself

4.4 ML Features (Per Transaction)

The real-time scoring model sees exactly 5 engineered features per transaction: `amount_ratio_to_threshold`, `is_cash`, `hour_of_day`, `day_of_week`, `is_transfer_out`.

Two additional features were deliberately excluded after being caught as unreliable — this is a load-bearing part of the project's actual engineering history, not a footnote:

- `account_age_days` was removed after discovering every simulated fraud account was freshly minted (age 0) while every legitimate account was pre-existing (age ≥ 31 days) — a data-leak artifact of how the simulator builds identities, not a genuine fraud signal. Left in, it would have both been trivially gameable by a launderer using an older account, and unfairly penalized any real customer who simply opened a new account.
- `velocity_10min` was removed after its variance was found to be nearly zero across both fraud and legit transactions alike (almost no account transacts twice within 10 minutes in this simulation), which caused feature standardization to amplify its rare exceptions into unstable, arbitrary score swings.

5. System Architecture

Technology	Role	Isolation
Docker	Isolates each branch's raw transaction data and local model training	Branch-local only
Kafka	Per-branch transaction streaming transport	Branch-local topics
PyTorch (logistic regression)	Per-branch real-time structuring score, every transaction	Branch-local model, weights shared via FL
Flower	Federated averaging (FedAvg) of model weights across branches	Weights only — never raw data
Neo4j	One shared, anonymized money-trail graph	Shared — tokens/metadata only, never PII
Redis	Shared whiteboard + event bus (scores, flags, evidence, verdicts, reports)	Shared — anonymized only
LangGraph + Claude	The autonomous multi-agent investigation pipeline	Operates only on anonymized graph/Redis data

The isolation boundary is precise, not "everything is isolated": raw transaction data, customer PII, and each branch's local training data stay branch-local. The graph, the whiteboard, and the agents are *intentionally* centralized — cross-branch tracing is structurally impossible otherwise. This is not a privacy compromise: only irreversible tokens and non-identifying metadata ever cross that line.

5.1 The Multi-Agent Pipeline

```
[Structuring Detection Model] --score above threshold--> [Money-Trail Agent]
        |--below threshold--> no action

[Money-Trail Agent] (tools: get_outgoing_txns, get_incoming_txns, check_convergence)
   Walks the money trail hop by hop, cycle-safe, time-windowed.
   Stops on principled conditions: convergence found, dead end, cycle detected,
   time window exceeded, or a high safety ceiling — never a fixed small hop cap.
        |--convergence found--> [Adversarial Verification]
        |--otherwise--> insufficient evidence, no verdict

[Adversarial Verification]
   Prosecutor Agent argues guilty from the evidence path.
   Defense Agent argues the most plausible innocent explanation.
   Judge Agent weighs both -> verdict + confidence + rationale.

[Judge: GUILTY, high confidence] --> [Label Generator] (fires immediately, fully autonomous)
                                       writes a new training label into every involved
                                       branch's local retrain buffer
                                  --> [Report Agent]
                                       drafts an investigation report, ends with a
                                       privacy attestation, awaits human review
                                            --> human approves/rejects (the ONLY human step)
```

Detection → escalation → tracing → adversarial verification → label injection → retraining all run with zero human input. The single human checkpoint in the entire pipeline is reviewing the drafted report before a case is formally closed — and that review does not gate the model feedback loop, which fires the moment a verdict is reached.

6. Scope

6.1 In Scope

- 3 simulated bank branches as isolated Docker containers, each with real Kafka streaming
- A hard PII-masking boundary enforced in code, not policy
- A genuinely trained (not hand-coded) per-branch real-time structuring model
- One shared, anonymized Neo4j graph for cross-branch money-trail tracing
- Federated learning (Flower/FedAvg) across the 3 branches' models
- A fully autonomous LangGraph agent pipeline: detection → tracing → adversarial verification → labeling → reporting
- One human checkpoint: report review/approval
- A configurable-hop-count structuring/layering scenario generator with ground-truth logging for evaluation (never used by the detection pipeline itself)
- An evaluation harness comparing 4 ablation conditions (adversarial verification on/off × FL feedback loop on/off)

6.2 Out of Scope (For Now)

- Real bank system integration
- Production-grade security hardening at scale

- More than 3 branches (sufficient to demonstrate the cross-branch benefit)
- A dashboard/frontend (Neo4j Browser + terminal logs serve the current demo)
- Two additional fraud typologies already identified as future work:
- **Single massive cash deposit** — catchable by real-time scoring alone, no graph tracing needed
- **Same individual depositing at all 3 branches same day** — catchable by behavioral similarity matching, not money-trail tracing Both are designed to be additive later without changing anything about the current architecture.

6.3 Session-Wise Build Plan

Session	Focus	Status
1	Foundation & scaffolding — Docker/Kafka/Redis/Neo4j skeleton	Complete
2	Data & scenario generation — realistic traffic + configurable-hop fraud scenario	Complete
3	Kafka streaming + masking + local ML scoring	**Complete — see §7 below for what was actually delivered, including two real issues found and fixed post-initial-pass**
4	Neo4j graph layer & convergence logic	Not started
5	Federated learning (Flower)	Not started
6	Agentic investigation pipeline (LangGraph core loop)	Not started
7	Adversarial verification + FL feedback loop	Not started
8	Report agent + human review gate + full integration	Not started
9	Evaluation harness + ablation runs + demo rehearsal	Not started

7. Progress So Far: Session 3 in Detail

This section exists because Session 3 didn't just meet its original exit criteria — it uncovered and fixed two genuine data-integrity problems worth documenting for anyone continuing this build.

What was built: a real-time pipeline where each branch independently masks, scores, and flags every transaction the instant it arrives — no batching, no delay.

The model: initially a small neural network trained on 2,000 synthetic examples built from a handwritten rule. This was replaced with a genuinely trained Logistic Regression, learned from 5,700+ transactions spanning 25 independently generated fraud scenarios (different people, hop counts, and placement amounts) plus 10 pure background-traffic batches — with a real held-out test set never touched during training.

Two real problems were found and fixed along the way, not glossed over: 1. A **data leak**: every simulated fraud account happened to be brand-new (age 0), while every legitimate account was pre-existing

— the model was learning "is this account new," not anything about the transaction itself. Fixed by removing the feature. 2. A **numerical instability**: a near-constant feature (transaction velocity) was having its rare exceptions amplified into unstable, outsized influence on the score by standardization. Fixed by removing the feature.

Final, honestly measured result (on the held-out test set, never seen during training):

- ROC-AUC: **0.935**
- At the chosen operating threshold (0.3): **100% recall on cash deposits/withdrawals**, ~93-95% recall on the harder mid-chain layering transfers, ~95-96% overall recall
- **Trade-off, stated plainly:** achieving that recall costs a ~27% false-positive rate on legitimate traffic. There is no threshold that avoids this trade-off — closing that gap is explicitly the job of Session 4's graph tracing (connecting transactions across accounts and branches), not a job for a better-trained single-transaction score.

Live confirmation: deployed to all 3 branch containers and run against the full demo scenario — **16/16 fraud transactions and 13/13 fraud accounts caught**, consistent with the offline test numbers (a good sign the model isn't overfitting to the test set).

8. Assumptions & Limitations

Assumptions:

- 3 branches are sufficient to demonstrate the federated/cross-branch benefit
- Synthetic data is statistically realistic enough to train and evaluate a meaningful structuring detector for this project's purposes
- Claude returns structured, parseable output reliably enough for agent reasoning, with fallback handling for the cases it doesn't

Limitations, stated honestly:

- Synthetic data cannot capture the full diversity of real-world laundering techniques — this system is validated against the specific structuring/layering pattern it was built to catch, not laundering in general
- The current model's honest ceiling (§7) reflects a genuine limitation of single-transaction scoring, not a tuning problem — this is a documented, expected result, not a shortcoming to be hidden
- A single-machine Docker setup does not reflect the latency/failure characteristics of true distributed branch infrastructure

9. Success Criteria (Full System, End State)

Metric	Target
Federated AUC vs. single-branch AUC	Federated higher by ≥5%
Cross-branch fraud detection rate	≥85% of injected scenarios caught end-to-end
Detection-to-report time	Under 2 minutes, live
Auto-generated investigation reports	100% of GUILTY verdicts
Zero PII in any shared payload	Verified by audit of every shared-layer write
FL model acceptance rate	>70% of federated rounds improve AUC
Label precision	Agent-verified labels validated against injected ground truth

This document reflects the project's actual state as of the completion of Session 3's rework. It will need updating as Sessions 4 onward land — see `SESSION_PLAN.md` *for the live build checklist and* `CLAUDE.md` *for the full technical source of truth.*